Model Context Protocol Server for Siemens NX CAD

Comprehensive Codebase Documentation

Generated: August 13, 2026
Version: 0.1.0 | Python 3.11 | NX 2206

Metric	Value
Source lines (nx_bridge.py)	~5,300
Total tool functions	~60
Tool modules	9
Test suite	51 tests
NX communication modes	2 (Socket + Batch)
Supported NX versions	NX 2206+

1. Overview

NX-MCP is a Model Context Protocol (MCP) server that bridges AI coding agents (such as Claude/Cline/Cursor) to Siemens NX CAD software. It allows AI agents to programmatically create parts, sketches, features, patterns, and assemblies by calling NX-MCP tools via JSON-RPC — without writing NXOpen journal scripts manually.

Key Capabilities

- Full solid modelling: cuboids, cylinders, extrusions, revolves, holes, fillets, chamfers
- Sketch-based workflow: create sketch planes, draw 2D profiles, extrude to 3D
- Parametric expressions: all dimensions are named NX expressions — editable after creation
- Pattern and mirror operations: circular/linear patterns, mirror body/feature
- Boolean operations: unite, subtract, intersect with real NXOpen Boolean builders
- Recipe execution: send multi-step feature sequences as a single JSON payload
- Journal code execution: run arbitrary NXOpen Python snippets inside live NX session
- Export: STEP and STL export directly from MCP tools
- Part inspection: list features, bodies, bounding box, and expressions
- Dual communication: interactive socket mode (live NX GUI) and batch subprocess mode

2. Architecture

NX-MCP uses a strict 3-layer architecture. Each layer has a single well-defined responsibility: protocol handling, tool logic, and NX execution.

Layer 1 — MCP Server

Files: server.py, http_server.py, response.py. Handles JSON-RPC 2.0 protocol over stdio or HTTP (FastAPI on port 8765). Routes tool/call requests to the ToolRegistry, returns ToolResult/ToolError responses.

Method	Purpose
`initialize`	Handshake — returns server capabilities and protocol version
`tools/list`	Returns all registered tools with JSON Schema input descriptors
`tools/call`	Executes a tool by name with given arguments

Layer 2 — Tool Layer

Files: tools/registry.py + 9 tool modules. Each tool is a Python function decorated with @mcp_tool(name, description). The ToolRegistry uses inspect.signature() to auto-generate JSON Schema inputSchema for each tool.

Module	Category	Example Tools
`file_ops.py`	File I/O	create_part, open_part, save_part, export_step, export_stl
`sketch.py`	Sketching	create_sketch, draw_line, draw_rectangle, draw_circle
`modeling.py`	Solid Modelling	create_cylinder, add_hole, add_hole_nx, add_fillet, pattern_circular, mirror_feature
`surface_modeling.py`	Surface/Sheet	create_base_surface, extrude_surface, thicken_sheet, offset_surface
`combine_operations.py`	Boolean/Trim	boolean_combine, trim_sheet, sew_sheets, split_body
`edit_operations.py`	Surface Editing	reverse_normal, x_form, i_form, heal_surface, match_edge
`assembly.py`	Assembly	add_component, list_components, mate_components
`measure.py`	Measurement	measure_distance, measure_angle, measure_volume, get_bounding_box
`utility.py`	Utilities	set_view, fit_view, list_features, inspect_part, reset_nx_bridge, run_recipe

Layer 3 — Bridge Layer

Files: bridge/runner.py (MCP server side) and bridge/nx_bridge.py (NX side). runner.py manages NX process lifecycle and serializes all bridge calls through a threading.Lock. nx_bridge.py runs inside Siemens NX's isolated Python runtime and is the only file that imports NXOpen.

Data Flow

```
AI Client  →  JSON-RPC tools/call {"name":"create_cylinder","arguments":{...}}
    ↓
server.py  →  ToolRegistry.get("create_cylinder")
```

```
    ↓
modeling.py create_cylinder()  →  runner.call_nx("create_cylinder", {...})
    ↓
runner.py  →  TCP socket port 43210  →  {"tool":"create_cylinder","args":{...}}
    ↓
nx_bridge.py dispatch()  →  create_cylinder()  →  NXOpen API calls
    ↓
Returns: {"ok": true, "message": "Cylinder R50 height 10 at (0,0,0)"}
    ↓ (back through all layers)
AI Client receives: OK: Cylinder R50 height 10 at (0,0,0)
```

3. Bridge Layer Deep Dive

3.1 Communication Modes

The bridge supports two modes. Interactive socket mode is always preferred. Batch mode is an emergency fallback only.

Mode	Transport	When Used	Limitations
`Interactive Socket`	TCP 127.0.0.1:43210	NX GUI is running with nx_bridge.py loaded	Requires NX to be started first (12s poll)
`Batch Subprocess`	stdin/stdout	No live socket available	No GUI, shorter timeout, state lost on restart

3.2 Anti-Duplicate Launch Guard

A critical bug was fixed where multiple parallel tool calls each independently detected 'no socket' and spawned their own ugraf.exe — producing multiple NX windows and router errors. The guard works in two steps:

- _find_running_ugraf() scans the OS process list for existing ugraf.exe before launching
- bridge.launching.lock (msvcrt.locking file mutex) ensures only one thread/process launches NX at a time
- Other callers block on the socket poll rather than spawning fresh instances

3.3 Socket Timeouts

Tool Group	Timeout	Env Override
`Standard tools (create_cube, add_fillet, …)`	30 seconds	NX_MCP_TIMEOUT
`Journal/Recipe tools (run_recipe, exec_python_code, run_nx_journal_code, execute_journal)`	120 seconds	NX_MCP_JOURNAL_TIMEOUT

3.4 NX Bridge Global State

nx_bridge.py maintains module-level globals that persist for the lifetime of the ugraf.exe process. These are reset to clean defaults by _reset_transient_session_state() on every create_part() and open_part() call.

```
ACTIVE_SKETCH_PLANE = "XY"      # reset to XY on create_part/open_part

_state = {
    "last_curves":    [],        # NX curves from latest sketch op
    "last_feature":   None,      # most recently created feature
    "last_body":      None,      # most recently created/modified body
    "features":       {},        # name → NXOpen.Feature (fuzzy-matched)
    "bodies":         {},        # name → NXOpen.Body (fuzzy-matched)
    "sketches":       {},        # name → NXOpen.Sketch
    "active_sketch":  None,      # currently active sketch object
    "sketch_origin":  [0,0,0],   # 3D origin for active sketch plane
    "sketch_rotation": 0.0,      # rotation angle for active sketch plane
}

_expression_counters = {}       # prefix → next suffix number
_instance_counters   = {}       # feature_type → next instance number
```

3.5 Fallback Part Safety

_ensure_work_part() is called by every tool before accessing the work part. It now checks whether the default fallback file (nx_mcp_result.prt) contains prior geometry, and creates a fresh nx_mcp_result_N.prt instead of

appending to old content:

```
if SESSION.Parts.Work is None:
    default_path = Path("nx_mcp_result.prt")
    if default_path.exists():
        SESSION.Parts.OpenDisplay(str(default_path))
        work_p = SESSION.Parts.Work
        # Guard: has prior geometry?
        if len(_iter_features(work_p)) > 0 or len(_iter_bodies(work_p)) > 0:
            counter = 1
            while True:
                candidate = f"nx_mcp_result_{counter}.prt"
                if not Path(candidate).exists():
                    SESSION.Parts.NewDisplay(str(candidate), ...)
                    break
                counter += 1
```

4. State Management

4.1 Feature and Body Resolution

Tools reference previously created geometry by string name. The bridge resolves these references through a priority-ordered lookup chain.

Reference Input	Resolution Strategy
`"last", "last body", "target"`	_state['last_body'] — most recently created body
`"body_1", "body_2"`	1-indexed lookup into _state['bodies'].values()
`Any other string`	_normalize_name() fuzzy match against _state['bodies'] keys
`Single body in part`	Auto-selects if only one body exists

4.2 _normalize_name()

Strips parenthetical suffixes so that 'CYLINDER(0)', 'Cylinder(0)', and 'cylinder' all resolve to the same feature. This fixed the 'Could not resolve feature reference' errors in mirror/pattern tools.

```
def _normalize_name(name: str) -> str:
    import re
    return re.sub(r'\s*\(\d+\)$', '', str(name)).strip().upper()

# Examples:
# "CYLINDER(0)"   → "CYLINDER"
# "Simple Hole(1)"→ "SIMPLE HOLE"
# "CUBOID_2"      → "CUBOID_2"
```

4.3 Edge Selection

Edge Reference	Selection Logic
`"all", "outer_edges"`	All edges on the body
`"top_edges"`	Edges at max-Z bounding position
`"bottom_edges"`	Edges at min-Z bounding position
`"vertical_edges"`	Edges aligned with Z axis (endpoint Z delta dominant)
`"x_aligned_edges"`	Edges aligned with X axis
`"y_aligned_edges"`	Edges aligned with Y axis
`"min_z_edges"`	Edges whose centroid Z equals body min Z
`"max_z_edges"`	Edges whose centroid Z equals body max Z

4.4 State Backup / Restore

Every command is wrapped with a state snapshot. On logical failure (ok=False) or exception, the snapshot is restored — preventing partial operations from corrupting subsequent tool calls.

5. Parametric Expression System

All dimensions are stored as named NX Expressions. This means users can later call edit_expression() to change a single value and NX will automatically rebuild all dependent geometry without re-running the full build sequence.

5.1 Expression Types

Type	Creator Function	Example
`Input`	_create_input_expression(wp, name, value)	CUBOID_1_L = 40.0
`Formula`	_create_formula_expression(wp, name, formula)	CUBOID_1_HALF_L = CUBOID_1_L / 2
`Named`	_create_named_expression(wp, prefix, value)	RADIUS_3 = 10.0 (auto-suffixed)

5.2 Instance Prefix System

Multiple features of the same type need unique expression names. _next_instance_prefix('CUBOID') returns 'CUBOID_1', then 'CUBOID_2', etc. On bridge restart, _seed_counters_from_part() scans existing part expressions so counters continue from where they left off.

```
prefix = _next_instance_prefix("HOLE")   # → "HOLE_1"
d_name    = _create_input_expression(wp, f"{prefix}_D",     10.0)  # HOLE_1_D = 10.0
dep_name  = _create_input_expression(wp, f"{prefix}_DEPTH", 15.0)  # HOLE_1_DEPTH = 15.0

# Next hole:
prefix2 = _next_instance_prefix("HOLE")  # → "HOLE_2"
```

5.3 Standard Builder Pattern

```
def create_something(param1, param2):
    work_part = _ensure_work_part()
    prefix = _next_instance_prefix("SOMETHING")

    # 1. Parametric expressions
    p1_name = _create_input_expression(work_part, f"{prefix}_P1", param1)
    p2_name = _create_input_expression(work_part, f"{prefix}_P2", param2)

    # 2. NX feature builder
    builder = work_part.Features.CreateSomeBuilder(NXOpen.Features.Feature.Null)
    try:
        builder.SomeProperty.RightHandSide = p1_name
        feature = builder.CommitFeature()
        _remember_feature(feature)
    finally:
        builder.Destroy()   # ALWAYS in finally - prevents resource leaks

    work_part.Save(NXOpen.BasePart.SaveComponents.TrueValue,
                   NXOpen.BasePart.CloseAfterSave.FalseValue)
    return {"ok": True, "message": f"...", "filepath": str(work_part.FullPath)}
```

6. Tool Reference

6.1 File Operations (file_ops.py)

Tool	Key Parameters	Description
`create_part`	filename	Create and open a new .prt file. Resets session state. Calls _reset_transient_session_state().
`open_part`	filepath	Open existing .prt. Resets session state. Seeds expression counters from part.
`save_part`	—	Save the current work part to disk.
`close_part`	save=True	Close the work part with optional save.
`export_step`	filepath	Export current part as STEP (.stp) via NXOpen.
`export_stl`	filepath, chord_tol	Export as STL mesh with given chord tolerance.
`list_parts`	—	List all parts currently open in the NX session.

6.2 Solid Modelling (modeling.py)

Tool	Key Parameters	Notes
`create_cube`	size, x, y, z	Creates cube via sketch+extrude. Fully parametric.
`create_cuboid`	length, width, height, x, y, z, centered	Box with separate L/W/H. centered=True places origin at center.
`create_cylinder`	radius, height, x, y, z, direction	Cylinder. direction: +Z/-Z/+X/-X/+Y/-Y.
`create_sphere`	radius, x, y, z	Sphere via revolve of semicircle profile.
`extrude`	distance, direction, sketch_name	Extrude named sketch or last sketch curves.
`revolve`	angle, axis, sketch_name	Revolve named sketch around axis.
`boolean_unite`	target, tool	Unite two bodies. 'last' resolves to last body.
`boolean_subtract`	target, tool	Subtract tool from target. Validates intersection first.
`add_hole`	x, y, z, diameter, depth, direction	High-level hole. Falls back to _subtract_cylinder_hole_parametric.
`add_hole_nx`	x, y, z, diameter, depth, direction	NX HoleBuilder-based hole. Pre-validates bounding box. Falls back to subtract.
`add_fillet`	radius, edges	Edge blend on named/top/all edges.
`add_chamfer`	distance, edges	Chamfer on named/top/all edges.
`pattern_circular`	feature, count, angle, axis, origin	Circular pattern using NXOpen PatternFeatureBuilder.
`pattern_linear`	feature, count, pitch, direction	Linear pattern along named direction.
`mirror_feature`	feature, plane	Mirror feature about named plane or XY/XZ/YZ.
`mirror_body`	body, plane	Mirror body. Creates new mirrored body.

6.3 Hole Validation (add_hole / add_hole_nx)

Both hole tools run a two-layer validation to prevent silent failures:

- Pre-check: _validate_hole_intersection() computes the hole cylinder's AABB and checks overlap with the target body's AABB (1mm margin). Returns 'Hole position does not intersect the target body - no material removed' immediately if no overlap.
- Post-check (_subtract_cylinder_hole_parametric): counts edges before and after subtract. If edge count did not increase, the hole missed the solid and returns the same error.

6.4 Utility Tools (utility.py)

Tool	Description
`list_features`	List all features in the current part by name and type
`inspect_part`	Full part inspection: features, bodies, bounding box, expressions, face count
`get_bounding_box`	Return min/max XYZ of a named body
`fit_view`	Fit the NX display to show all geometry
`set_view`	Set NX view orientation (TOP, FRONT, RIGHT, ISO, etc.)
`take_screenshot`	Capture NX viewport as PNG
`reset_nx_bridge`	Kill ugraf.exe and run_journal.exe; delete live_bridge_config.json
`get_relevant_nx_tools`	Search tool registry by description keyword — reduces tool list for AI
`run_recipe`	Execute a list of {tool, args} steps sequentially in one call
`exec_python_code`	Execute arbitrary Python code inside live NX session (120s timeout)
`run_nx_journal_code`	Run NXOpen journal Python in NX (120s timeout)

7. Workflow Guide

7.1 AI Client Workflow Rules (.clinerules)

The .clinerules file defines the mandatory workflow for AI agents using NX-MCP. Key rules in order of priority:

- COMPLEXITY CHECK: If the part needs computed curves (involute gears, splines, helical paths), go directly to run_nx_journal_code with a complete script.
- 0. ALWAYS call get_relevant_nx_tools with a task description FIRST to narrow down the tool list.
- 0.5. ALWAYS call create_part with a descriptive filename before building anything. Never rely on the fallback nx_mcp_result.prt file being empty.
- 1. Decompose the part into an ordered feature list (block/sketch/extrude/cut/hole/pattern/mirror/blend).
- 2. Define all driving dimensions first with create_expression.
- 3. PREFER run_recipe: send the whole feature list as a single steps payload for deterministic execution.
- 4. Verify with inspect_part after building.
- 5. If a journal script succeeds for a new part family, save it to src/nx_mcp/journals/templates/.py.

7.2 Standard Part Build Sequence

```
# Step 1: Create a named fresh part
create_part("bracket_v1.prt")

# Step 2: Create driving expressions
create_expression("L", 100)
create_expression("W", 60)
create_expression("H", 20)

# Step 3: Build geometry with run_recipe
run_recipe(steps=[
    {"tool": "create_cuboid", "args": {"length": "L", "width": "W", "height": "H"}},
    {"tool": "add_hole",      "args": {"x": 0, "y": 0, "z": "H", "diameter": 10, "depth": "H", "direction": "
    {"tool": "add_fillet",    "args": {"radius": 2, "edges": "top_edges"}},
])

# Step 4: Inspect result
inspect_part()
```

7.3 Error Scenario Handling

Error	Source	Action
`File already exists`	create_part	Pick a different filename or open_part on same path
`Hole position does not intersect body`	add_hole / add_hole_nx	Check x/y/z coordinates vs body bounding box
`Could not resolve feature reference`	mirror/pattern	Use exact name from list_features output
`Batch mode failed/timed out`	runner.py	reset_nx_bridge() then retry; NX may need restart
`Router Error - Failed to communicate`	ugraf.exe	Stale ugraf.exe — kill all ugraf.exe, delete bridge.lock

8. Bug Fix History

This section documents critical bugs discovered and fixed during development, along with root causes and the implemented solutions.

BUG-01: Duplicate NX Launch Race Condition

Symptom: Multiple ugraf.exe windows spawned simultaneously, each throwing 'Router Error - Failed to communicate with the other application'

Root Cause: Parallel tool calls each detected 'no socket' and independently called launch_interactive_nx() without checking if a launch was already in progress

Fix: _find_running_ugraf() OS-level process scan + bridge.launching.lock file mutex (msvcrt.locking). Other callers wait on socket poll instead of launching.

BUG-02: Feature Reference Resolution Failures

Symptom: mirror_feature, mirror_body_feature, pattern_feature, and pattern_circular all failed with 'Could not resolve feature reference' even when passing valid feature names from list_features

Root Cause: NX feature names include parenthetical suffixes like '(0)' or '(1)' that were not stripped during fuzzy matching. 'HOLE_1' did not match 'Simple Hole(1)'.

Fix: _normalize_name() strips trailing (N) suffixes. Applied in _resolve_feature(), _resolve_body(), and all scanner loops.

BUG-03: pattern_circular AngularSpacing API Mismatch

Symptom: pattern_circular timed out or produced no output on plate builds

Root Cause: NCopies and PitchAngle were accessed directly on circ rather than through circ.AngularSpacing. Axis was passed as Vector3d instead of required NXOpen.Point SmartObject.

Fix: Fixed property paths to circ.AngularSpacing.NCopies and circ.AngularSpacing.PitchAngle. Added _create_axis() helper that constructs NXOpen.Point and NXOpen.Direction SmartObjects correctly.

BUG-04: Wrong Part Bug — Building onto Stale Fallback File

Symptom: New part geometry was added onto an old nx_mcp_result.prt file that already contained a previous part's full feature history

Root Cause: _ensure_work_part() silently opened nx_mcp_result.prt if it existed, without checking whether it already contained geometry from a previous session

Fix: _ensure_work_part() now checks feature/body count after opening the fallback. If geometry exists, it creates a fresh nx_mcp_result_N.prt instead.

BUG-05: Stale Sketch State Leak Between Part Builds

Symptom: Holes and features on a new part were positioned incorrectly, offset by the origin/rotation of a sketch created during the previous build

Root Cause: ACTIVE_SKETCH_PLANE, sketch_origin, and sketch_rotation in nx_bridge.py persisted across independent part builds because ugraf.exe stays alive between sessions

Fix: _reset_transient_session_state() added, called in create_part() and open_part(). Resets plane, origin, rotation, and all feature/body/sketch caches.

BUG-06: Invisible Holes — Silent Subtraction Miss

Symptom: add_hole_nx called 8 times, all returned 'OK', but zero holes were visible on the finished part

Root Cause: NXOpen BooleanSubtract consumed the tool body without removing material when the tool body did not intersect the target. No error was raised.

Fix: _validate_hole_intersection() bounding box pre-check + post-subtract edge count comparison. Returns 'Hole position does not intersect the target body - no material removed' on miss.

BUG-07: run_recipe Timeout on pattern_circular

Symptom: run_recipe building a plate (cylinder + holes + circular pattern + fillet) timed out

Root Cause: run_recipe used the short NX_MCP_TIMEOUT instead of NX_MCP_JOURNAL_TIMEOUT. Combined with pattern_circular API bugs (BUG-03), execution exceeded the 30s window.

Fix: Added 'run_recipe' to the extended-timeout tool set in _call_socket(). Fixed pattern_circular API paths (BUG-03). Recipe now executes in ~1.64 seconds.

9. Testing

9.1 Unit Test Suite

The test suite in tests/ runs without a live NX installation using mock_nxopen. All 51 tests pass in ~0.20 seconds.

```
python -m pytest tests/ -v
```

9.2 Live Verification

verify_three_fixes_live.py validates all three architectural fixes against a live NX session:

Scenario	What It Tests	Expected Result
`A — State Isolation`	create_part(B) after contaminating sketch state in Part A	[PASS] Hole lands on correct XY plane at origin
`B — Fallback Part Safety`	Trigger _ensure_work_part() after nx_mcp_result.prt has geometry	[PASS] Fresh nx_mcp_result_N.prt created
`C — Hole Miss Validation`	add_hole_nx at (500, 500) on 100×100×10mm cuboid	[PASS] Error: 'no material removed'

9.3 Environment Setup

```
# Install dependencies
pip install -e .

# Set NX path (required for live tests)
$env:UGII_BASE_DIR = "C:\Program Files\Siemens\NX2206"

# Run unit tests (no NX required)
python -m pytest tests/

# Run live verification (NX must be running)
python verify_three_fixes_live.py
```

10. Configuration Reference

10.1 Environment Variables

Variable	Default	Description
`UGII_BASE_DIR`	(required)	Path to Siemens NX installation root. Must contain NXBIN/run_journal.exe
`NX_MCP_ROOT`	Auto-detect	Override project root directory
`NX_MCP_USE_MOCK_NXOPEN`	0	Set to '1' to use mock NXOpen for testing without NX installed
`NX_MCP_TIMEOUT`	30	Default socket timeout in seconds for standard tools
`NX_MCP_JOURNAL_TIMEOUT`	120	Extended socket timeout for run_recipe, exec_python_code, etc.
`LIVE_PORT`	43210	TCP port for NX socket bridge communication

10.2 live_bridge_config.json

Written by runner.py when launching NX in interactive mode. Read by nx_bridge.py on startup to configure the socket port. Deleted by reset_nx_bridge() to signal graceful shutdown.

```
{"part_path": "C:/Users/HP/nx-mcp/nx_mcp_result.prt", "port": 43210}
```

10.3 MCP Client Configuration (Claude Desktop)

```
// claude_desktop_config.json
{
  "mcpServers": {
    "nx-mcp": {
      "command": "python",
      "args": ["-m", "nx_mcp.server"],
      "cwd": "C:/Users/HP/nx-mcp",
      "env": {
        "UGII_BASE_DIR": "C:/Program Files/Siemens/NX2206"
      }
    }
  }
}
```

11. Journal Templates

src/nx_mcp/journals/templates/ contains reusable NXOpen Python scripts for complex part families that cannot be built with primitive tool chaining (gear teeth, helical paths, complex swept profiles). The AI agent checks this directory before building a similar part.

Template File	Part Type	Key NXOpen APIs
`spur_gear.py`	Involute spur gear	CreateExtrudeBuilder, AngularSpacingBuilder, involute math
`clevis_mount.py`	Clevis/fork bracket	CreateCylinderBuilder, CreateHoleBuilder, BooleanSubtractBuilder

Adding a new template: when run_nx_journal_code succeeds for a new part family, call write_to_file to save the script at src/nx_mcp/journals/templates/.py for future reuse.